

Suggested ChurchManager Hymnal Import Process

Status: Proposed; awaiting approval

Prepared: August 15, 2026

Depends on: Permanent Hymn Identifier Specification

1. Purpose

Provide a safe, repeatable process for adding a hymnal to ChurchManager without creating identifier collisions, changing historical worship records, importing unlicensed content, or making installation order significant.

ChurchManager will support two deliberately different workflows:

1. **Curated hymnal package import** - an approved, reusable catalog with a permanent HymnalID and permanent hymn-number block.
2. **Local congregation import** - user-supplied hymn metadata placed only in the reserved local-hymn range.

A local import can never claim or manufacture official package IDs.

2. Phase A: approve the source and scope

Before preparing data:

1. Identify the hymnal, edition, publisher, publication year, and ISBN or other edition identifier.
2. Record where the proposed data came from and who prepared it.
3. Determine what ChurchManager may legally redistribute.
4. Exclude lyrics, copyrighted music images, harmonizations, recordings, and other protected content unless written permission or a compatible license is documented.
5. Define the approved metadata fields for that source.

Ordinarily safe catalog metadata may include:

- printed hymn number or reference;
- title or first line;
- tune name;
- meter;
- topical category;
- Scripture references;
- author, translator, and composer names where legally appropriate;
- public-domain or copyright-status notes; and
- internal source and verification notes.

Copyright status must not be guessed merely from age or internet availability.

3. Phase B: reserve the permanent catalog block

Before assigning hymn IDs:

1. Add the hymnal to the maintained permanent catalog registry.
2. Assign one unused permanent HymnalID.
3. Reserve its corresponding 5,000-number block.
4. Record permanent HymnIDStart and HymnIDEnd values on the hymnal registry record.
5. Record a unique textual package code such as Lsb as secondary metadata.
6. Record the initial package version.

Once assigned, neither the HymnalID nor its block may ever be assigned to a different hymnal or edition.

Materially different editions should normally receive separate registry entries when their numbering or content differs enough to make identity ambiguous.

4. Phase C: prepare source data outside the live tables

Source data is prepared as UTF-8 CSV files and validated before it is allowed into ChurchDB.

Recommended package contents:

```
manifest.json
hymns.csv
README.md
LICENSE-or-SOURCE-NOTICE.txt
checksums.sha256
```

Required hymns.csv columns:

Column	Purpose
HymnID	Permanent assigned numeric identifier
HymnalID	Permanent hymnal registry identifier
EntrySlot	Slot 1 through 4,999 within the hymnal block
PrintedReference	What users see, such as LSB 656
Title	Hymn title or approved first line
IsActive	Whether the entry is available for new selection

Optional approved columns may include Tune, Meter, Category, ScriptureReferences, Author, Composer, CopyrightNote, and Note.

Full lyrics and binary files are not part of the ordinary hymnal package.

5. Phase D: normalize and assign permanent IDs

The package preparer works in a staging tool, not directly in tblHymn.

For each entry:

1. Normalize spaces, punctuation, capitalization, and Unicode characters without changing the printed title's meaning.

2. Preserve the printed reference separately from the database ID.
3. Use the printed hymn number as `EntrySlot` when it is a unique integer in the allowed range.
4. Assign a documented unused slot for duplicate, lettered, unnumbered, or exceptional entries.
5. Calculate and store the permanent HymnID from the approved block and slot.
6. Detect duplicate IDs, printed references, and suspiciously duplicated titles.
7. Flag missing titles, malformed references, and conflicting tune information for human review.

The import tool proposes values but does not silently resolve ambiguous records.

6. Phase E: validate the package

Automated preflight must reject the package when:

- HymnalID is not registered;
- the package range does not exactly match the range stored for that hymnal;
- the declared range overlaps another `tblHymnal` record;
- the package code or block does not match the registry;
- an `EntrySlot` is zero, negative, above 4,999, or duplicated;
- a HymnID does not equal the registered block plus its slot;
- a HymnID lies outside the assigned block;
- required fields are blank;
- two rows claim the same printed identity without an approved exception;
- a packaged ID conflicts with an existing different hymn;
- the declared dependency or version is missing;
- checksums fail; or
- required source or licensing documentation is absent.

Preflight produces a readable report containing:

- total source rows;
- accepted rows;
- warnings;
- rejected rows and reasons;
- assigned exceptional slots;
- detected duplicates; and

- package checksum.

Warnings require explicit approval before package publication.

7. Phase F: build the distributable package

After human review:

1. Freeze the normalized CSV.
2. Record package version and compatible ChurchManager schema version.
3. Generate file checksums.
4. Include source and licensing notices.
5. Sign or record an approved checksum for the package.
6. Store the approved package in the ChurchManager catalog source directory.
7. Preserve the preparation report with the project documentation.

Proper hymn suggestions and Orders of Service are separate packages. They may declare the hymnal package as a dependency and reference its permanent hymn IDs, but they are not silently bundled into the hymnal catalog.

8. Phase G: preview installation

The ChurchManager installer shows:

- hymnal title and edition;
- package version;
- source and license summary;
- number of new, existing, updated, and retired entries;
- dependencies;
- collisions or blocking errors; and
- whether this hymnal may become the church's primary hymnal.

The user must explicitly approve installation. Preview performs no database writes.

9. Phase H: transactional installation

After approval, ChurchManager:

1. Creates a safety backup.
2. Begins one database transaction.
3. Revalidates the registry, package checksum, range, and dependencies.
4. Inserts or verifies the permanent `tblHymnal` record.
5. Verifies and locks the hymnal's permanent `HymnIDStart` and `HymnIDEnd`.

6. Inserts new hymn records using their explicit permanent IDs.
7. Updates only metadata fields the package is authorized to maintain.
8. Preserves local notes and local extensions in separately owned fields.
9. Retires package entries marked inactive rather than deleting them.
10. Installs separately approved dependent packages only after all hymn records exist.
11. Verifies every foreign key and expected row count.
12. Records the installed package version and result.
13. Commits only if every step succeeds; otherwise rolls back everything.

The installer never disables foreign-key checks merely to force a package into the database.

10. Phase I: post-install verification

After commit, verify:

- package and database row counts agree;
- every hymn lies in the correct block;
- all IDs and printed references are unique as expected;
- representative first, middle, last, and exceptional entries display correctly;
- hymn search and column sorting work;
- the hymnal filter works;
- a hymn can be selected for a test service;
- Proper suggestions resolve correctly when their package is installed;
- reports display printed references rather than internal IDs; and
- backup and restore preserve the installed catalog.

The installation log records the package checksum, version, counts, warnings, installer user, and timestamp without recording database credentials.

11. Package upgrades

An upgrade uses the same preflight, preview, transaction, and verification flow.

An upgrade may:

- add entries in unused slots;
- correct approved metadata;
- add tune or Scripture-reference metadata;
- retire an entry; or

- reactivate a previously retired entry.

An upgrade may not:

- change an existing HymnID;
- move an entry to another hymnal block;
- reuse a retired ID;
- replace one hymn's identity with another;
- overwrite congregation-owned notes; or
- physically delete worship-history references.

Any proposed identity correction that truly requires a different ID must be a separate, reviewed data-correction migration with an explicit audit trail.

12. Local congregation imports

Local CSV imports use a simpler screen but retain strict safeguards.

1. The user selects a UTF-8 CSV file.
2. ChurchManager previews and maps columns.
3. It detects possible duplicates against both official and local catalogs.
4. The user reviews every ambiguous match.
5. New records receive the next never-used ID from 10,001 through 14,999.
6. Imported records are assigned to the local-user hymnal catalog.
7. The import commits transactionally and produces a result log.

Local imports may not supply HymnID, HymnalID, or EntrySlot. Those fields are assigned by ChurchManager. A local import cannot modify an official packaged hymn.

13. Recovery and reversibility

- A failed import rolls back automatically.
- An installed package is not casually "uninstalled" when its hymns are used by worship history.
- An administrator may deactivate an entire hymnal for future selection while retaining all historical records.
- Recovery from corruption uses the pre-install backup or complete backup, not ID regeneration.
- Package files and import logs are retained according to the backup and support policy.

14. Acceptance criteria

The process is ready for implementation when:

- the permanent block registry is approved;

- required and optional metadata fields are approved;
- copyright-review responsibilities are assigned;
- package and manifest schemas are written;
- staging, preflight, preview, install, and rollback behavior are specified in testable terms;
- the LSB dataset passes the full process as the reference package; and
- a local CSV import test proves that official IDs cannot be claimed or changed.